EECS370 Final

pipeline

IRON LAW: $t = \#instr \times CPI \times T$

processor
- single-cycle
- multi-cycle
- pipeline

multicycle

Fetch → Decode → ALU Execution → Register Write (add, nand) / Memory Access (sw, lw) / PC += offset (beq)

single cycle

- mem access ×2
- reg read + write
- ALU

pipeline

① IF	② ID	③ EX	④ MEM	⑤ WB
instr fetch	reg read	execution	branch resolved	mem access done

(lw available for forwading)

hazard
- data ~
- control ~
- structural ~

Data Hazard

*occur within 2 instruction window if internal forwarding reg.

Detect & Stall

add/nand/lw 后 noop ×2

Detect & Forward

从各reg forward 到 EX

add/nand 不用 noop

lw 后 noop ×1 (除 lw 了 ⑥, 特例)

sw × ⑥ ×

Control Hazard (beq)

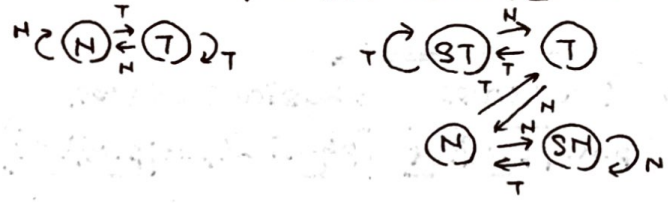
Detect & Stall 3 noop

Speculate & Squash 3 noop if wrong

(instr left in IE, noop sent to ID.EX.MEM)

branch prediction

- static
  - always taken / not taken
  - BTFN (backward T, forward N)
- dynamic
  - last time prediction
  - 2-bit counter ~



cache

- temporal locality – LRU/LFU/FIFO
- spacial locality

avg latency = cache latency + memory latency × miss rate

- write through
- write back (dirty bit)

allocate on write

- fully-associative #set = 1
- directly-mapped #way = 1
- set-associative

address

tag	set index	block offset

cache size = blocksize × #blocks (always 2^n)

#sets × #ways

addr = tag + bit blockoffset + bit set idx

overhead storage = tag + valid + dirty

3Cs cache misses

- compulsory ~ : cold start
- capacity ~ : hit if larger cache
- conflict ~ : hit if higher associativity

* simulate with ∞ cache
* simulate with fully associative cache
* simulate with intended cache.

virtual memory

* transparency * protection

cache (SRAM)
main memory (DRAM)
disk

single level page table

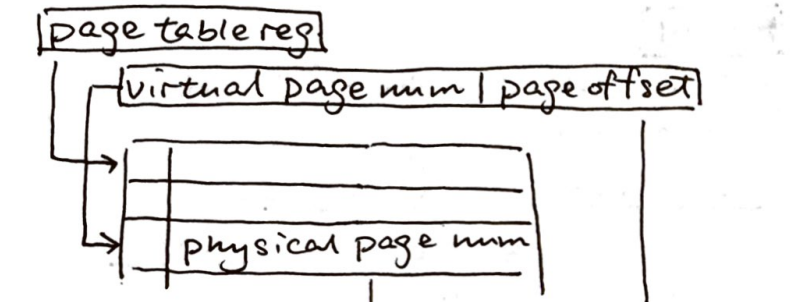
virtual address: | virtual p.n. | p.offset |

physical address: | physical p.n | p.offset |

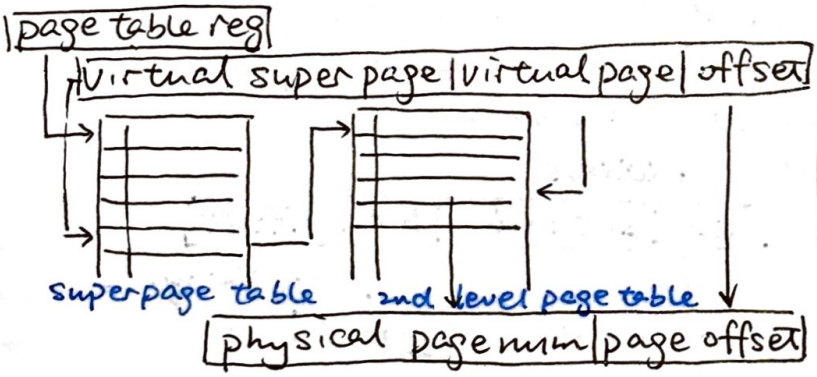
0	0000	8	1000
1	0001	9	1001
2	0010	A	1010
3	0011	B	1011
4	0100	C	1100
5	0101	D	1101
6	0110	E	1110
7	0111	F	1111

$2^0=1$	$2^6=64$
$2^1=2$	$2^7=128$
$2^2=4$	$2^8=256$
$2^3=8$	$2^9=512$
$2^4=16$	$2^{10}=1024$
$2^5=32$	

K: 2^{10} M: 2^{20} G: 2^{30}



multi level page table (size ∝ mem used)



TLB

V	tag	physical page

cache

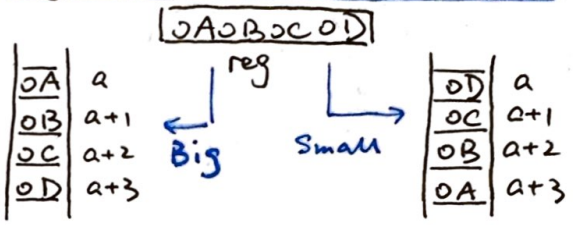
- physically addressed (slow)
- virtually addressed (complex)
  - * perform TLB lookup at the same time as the cache tag compare

problems
- * same v. addr → diff p. addr
- * diff v addr → same p. addr

⇒ cache flushed between context switch

processID as part of tag

Big Endian vs. Small Endian



Data Layout

- *p 4B
- double 8B
- float 4B
- int 4B
- short 2B
- char 1B

Object Code

- Header
- Text
- Data
- Symbol Table
- Debug Info

LC2K

- *32bit reg ×8
- *65536 words mem
- *1 word = 4 byte

Numbers

Negative: 2's Complement

bitwise neg +1 ↔

Floating

sign: 0 +; 1 −

mantissa: 23 bits after "."

exponent: biased 127

encoding 8 bits

e.g. −127 → 00000000

−126 → 00000001

0 → 01111111

1 → 10000000

2 → 10000001

128 → 11111111

e.g. $10.625_{10} \Rightarrow 1010.101_2$

$\Rightarrow 1.010101 \times 2^3$

⇒ 1 | 10000101 | 01011001...